

ANKARA ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ



Proje Adı: Döküman Yönetim Sistemi (DMS)

Öğrenci: Dursun Can Çınar

Öğrenci No: 22290285

Ders: Ağ Tabanlı Teknolojiler ve Uygulamaları

GitHub Proje Linki: https://github.com/drsncan/Dursun_Can_Cinar_WEB

Video Linki: <https://youtu.be/IIHZu3TPz3w>

1. Projenin Amacı

Bu projenin amacı, kurum içi doküman süreçlerinin dijital ortamda etkin, güvenli ve kontrollü bir şekilde yönetilebilmesini sağlayan web tabanlı bir **Belge Yönetim Sistemi (Document Management System – DMS)** geliştirmektir.

Geliştirilen sistem sayesinde kullanıcılar belge yükleyebilmekte, yüklenen belgelerin durumları (*beklemede, onaylandı, reddedildi*) takip edilebilmekte ve yetkili kullanıcılar tarafından onay süreçleri yönetilebilmektedir. Böylece kurum içi belge akışı düzenli, izlenebilir ve denetlenebilir bir yapı kazanmıştır.

Proje; belge güvenliğini artırmayı, manuel belge süreçlerinden kaynaklanan hataları azaltmayı ve kullanıcı dostu bir arayüz ile belgelerin kolay yönetilmesini hedeflemektedir. Aynı zamanda modern web mimarisi kullanılarak API, veritabanı ve kullanıcı arayüzü arasındaki veri akışı doğru bir şekilde kurgulanmıştır.

Bu kapsamda proje aşağıdaki temel özellikleri içermektedir:

- Kullanıcı ve rol yönetimi
- Belge yükleme ve onay mekanizması
- RESTful API entegrasyonu
- Modern ve kullanıcı dostu arayüz (UI)

2.Proje Kapsamı

Bu proje kapsamında, kullanıcıların sisteme giriş yaptıktan sonra sahip oldukları roller doğrultusunda yetkilendirildiği ve belge işlemlerini bu yetkilere göre gerçekleştirebildiği bir yapı oluşturulmuştur. Sistem, rol bazlı erişim kontrolü ile güvenli bir çalışma ortamı sunmaktadır.

Proje kapsamına dahil edilen temel özellikler aşağıda listelenmiştir:

- Kullanıcı kayıt ve giriş işlemleri (ASP.NET Identity tabanlı)
- Rol bazlı yetkilendirme mekanizması (*Admin, Kullanıcı, Onaycı*)
- Belge yükleme, silme ve görüntüleme işlemleri
- Belge durumlarının yönetimi (*Beklemede, Onaylandı, Reddedildi*)
- Belgeler için açıklama ekleme ve metadata kaydı (*yükleme tarihi, yükleyen kullanıcı, belge durumu*)
- Belge listeleri üzerinden filtreleme ve görüntüleme imkânı sağlayan basit raporlama özellikleri

Bu kapsam doğrultusunda geliştirilen sistem, kurum içi belge süreçlerinin kontrollü, izlenebilir ve düzenli bir şekilde yürütülmesini sağlamaktadır.

3.Kullanılan Teknolojiler

Katman	Teknoloji	Açıklama
Backend	ASP.NET Core 8 (MVC)	Projenin sunucu tarafında kullanılmıştır. Katmanlı mimari (Controller, Model, View) yapısı ile hem API hem de web arayüzü bu framework üzerinde geliştirilmiştir.
API	RESTful API	Frontend ile backend arasındaki veri iletişimi REST prensiplerine uygun olarak JSON formatında sağlanmıştır. CRUD işlemleri HTTP metotları (GET, POST, PUT, DELETE) ile yapılmaktadır.
ORM	Entity Framework Core 8	Veritabanı işlemleri için kullanılmıştır. SQL sorguları yazmak yerine C# nesneleri üzerinden veritabanı işlemleri yapılmasını sağlamıştır.
Veritabanı	SQL Server	Kullanıcılar, belgeler ve roller gibi veriler ilişkisel olarak saklanmıştır. EF Core ile entegre çalışmaktadır.
Frontend	HTML, CSS, Bootstrap 4.6	Kullanıcı arayüzü oluşturmak için kullanılmıştır. Bootstrap sayesinde responsive ve düzenli bir tasarım elde edilmiştir.

UI Template	AdminLTE	Yönetim paneli görünümü için hazır admin panel teması kullanılmıştır. Daha profesyonel bir arayüz sağlanmıştır.
JavaScript	jQuery & Fetch API	Sayfa yenilenmeden API çağrıları yapmak ve dinamik veri göstermek için kullanılmıştır.
Dosya Yönetimi	wwwroot/uploads	Yüklenen belgeler sunucu tarafında bu klasörde saklanmaktadır.
Versiyon Kontrol	Git & GitHub	Projenin sürüm takibi, yedeklenmesi ve paylaşımı için kullanılmıştır.
Yapay Zeka	ChatGPT	API taslakları, hata analizi ve kod iyileştirme önerileri için destek amaçlı kullanılmıştır. Kodlar manuel olarak düzenlenmiştir.

3. Sistem Mimarisi

Bu projede sistem mimarisi; **Frontend (Kullanıcı Arayüzü)**, **Backend (Uygulama ve API Katmanı)** ve **Veritabanı** olmak üzere üç ana katmanlı yapı şeklinde tasarlanmıştır. Katmanlar birbirinden bağımsız çalışmakta olup, aralarındaki iletişim belirli kurallar çerçevesinde sağlanmaktadır.

3.1 Frontend (Kullanıcı Arayüzü)

Frontend katmanı, kullanıcıların sistemle etkileşime geçtiği arayüzdür. Bu katmanda kullanıcılar:

- Sisteme giriş yapabilir,
- Belgeleri görüntüleyebilir,
- Yetkilerine bağlı olarak belge yükleme, silme ve onaylama işlemlerini gerçekleştirebilir.

Frontend tarafında yapılan işlemler doğrudan veritabanına erişmez. Tüm veri alışverişi **API üzerinden** gerçekleştirilir. Böylece güvenlik ve veri bütünlüğü sağlanmıştır.

3.2 Backend (Uygulama ve API Katmanı)

Backend katmanı, sistemin iş mantığının bulunduğu bölümdür. Bu katman:

- Kullanıcı yetkilendirme kontrollerini yapar,
- Gelen isteklerin doğruluğunu denetler,
- İş kurallarını uygular,
- Veritabanı ile iletişimi yönetir.

Backend içerisinde RESTful API yapısı kullanılmıştır. Frontend'den gelen HTTP istekleri Controller'lar tarafından karşılanır, gerekli işlemler yapıldıktan sonra sonuçlar JSON formatında frontend'e geri döndürülür.

Bu yapı sayesinde:

- Kodun okunabilirliği artmıştır,
- İş mantığı merkezi bir noktada toplanmıştır,
- Uygulama daha sürdürülebilir hale gelmiştir.

3.3 Veritabanı Katmanı

Veritabanı katmanı, sistemde kullanılan tüm kalıcı verilerin saklandığı bölümdür. Kullanıcılar, belgeler, roller ve belge durumları gibi bilgiler ilişkisel veritabanı yapısı kullanılarak tutulmaktadır.

Veritabanı işlemleri doğrudan SQL sorguları ile değil, **Entity Framework Core** aracılığıyla gerçekleştirilmektedir. Bu sayede:

- Veritabanı işlemleri daha güvenli hale gelmiştir,
- Kod tarafında veritabanı bağımlılığı azaltılmıştır,
- Bakım ve geliştirme süreçleri kolaylaştırılmıştır.

3.4 Katmanlar Arası Veri Akışı

Sistemde veri akışı aşağıdaki adımlar izlenerek gerçekleşmektedir:

1. Kullanıcı frontend üzerinden bir işlem başlatır (örneğin belge listeleme).
2. Frontend, ilgili API endpoint'ine HTTP isteği gönderir.
3. Backend, isteği Controller üzerinden karşılar.
4. Gerekli iş mantığı uygulanır ve veritabanından veri alınır.
5. Alınan veri JSON formatında frontend'e döndürülür.
6. Frontend, gelen veriyi kullanıcı arayüzünde görüntüler.

Bu mimari yapı sayesinde sistem:

- Güvenli,
- Modüler,
- Ölçeklenebilir bir hale getirilmiştir.

4. Kod Yapısı ve Katmanların Detaylı Açıklaması

Bu projede sürdürülebilirlik, okunabilirlik ve bakım kolaylığı sağlamak amacıyla **katmanlı mimari** yaklaşımı benimsenmiştir. Kod yapısı; sorumlulukların ayrıldığı, her katmanın belirli bir görevi üstlendiği şekilde tasarlanmıştır.

4.1 Controllers Katmanı

Controllers klasörü, frontend'den veya API üzerinden gelen HTTP isteklerini karşılayan yapıları içermektedir. Bu katman, kullanıcıdan gelen talepleri alır, gerekli kontrolleri yaptıktan sonra ilgili iş mantığını çalıştırır ve sonucu frontend'e iletir.

Controller'lar:

- API endpoint'lerini barındırır
- HTTP metotları (GET, POST, PUT, DELETE) ile çalışır
- Doğrudan veritabanı işlemi yapmaz

Bu yapı sayesinde uygulamanın kontrol akışı merkezi bir noktada yönetilmektedir.

4.2 Models Katmanı

Models klasörü, uygulamada kullanılan veri yapılarını temsil eden sınıfları içermektedir. Bu sınıflar, veritabanındaki tabloların nesne karşılıklarıdır.

Models katmanı:

- Kullanıcı, belge ve rol gibi temel varlıkları temsil eder

- Entity Framework Core ile veritabanı tablolarına karşılık gelir
- Veri bütünlüğünü sağlamak amacıyla gerekli alan tanımlarını içerir

Bu katman sayesinde uygulama ile veritabanı arasında güçlü ve tutarlı bir bağ kurulmuştur.

4.3 Data Katmanı

Data klasörü, veritabanı bağlantısının ve Entity Framework Core yapılandırmasının yapıldığı katmandır. Bu katmanda yer alan **DbContext** sınıfı aracılığıyla uygulama ile veritabanı arasındaki iletişim sağlanmaktadır.

Data katmanı:

- Veritabanı bağlantı ayarlarını içerir
- Tabloların uygulama içerisindeki karşılıklarını yönetir
- Migration işlemleri ile veritabanı şemasının güncellenmesini sağlar

Bu yaklaşım sayesinde veritabanı işlemleri daha güvenli ve yönetilebilir hale gelmiştir.

4.4 Views Katmanı

Views klasörü, kullanıcıya gösterilen arayüz sayfalarını içermektedir. Bu katman, Razor View yapısı kullanılarak geliştirilmiştir.

Views katmanı:

- Kullanıcı etkileşimlerini görsel olarak sunar
- Backend'den gelen verileri kullanıcıya gösterir
- HTML, CSS ve JavaScript ile desteklenmiştir

Bu yapı sayesinde kullanıcı dostu ve düzenli bir arayüz elde edilmiştir.

4.5 wwwroot Katmanı

wwwroot klasörü, uygulamada kullanılan statik dosyaların bulunduğu alandır. Bu klasörde CSS, JavaScript, görseller ve kullanıcı tarafından yüklenen belgeler yer almaktadır.

wwwroot katmanı:

- Uygulamanın görsel ve işlevsel kaynaklarını içerir
- Yüklenen belgelerin fiziksel olarak saklandığı dizindir
- Frontend performansını destekler

4.6 Katmanlı Yapının Sağladığı Avantajlar

Bu şekilde oluşturulan katmanlı mimari sayesinde:

- Kod okunabilirliği artırılmıştır
- Bakım ve geliştirme süreçleri kolaylaştırılmıştır
- Güvenlik ve veri bütünlüğü sağlanmıştır
- Frontend, backend ve veritabanı birbirinden bağımsız hale getirilmiştir

Bu yapı, projenin hem akademik hem de gerçek dünya uygulamalarına uygun bir mimariye sahip olmasını sağlamıştır.

5. API Yapısı ve Çalışma Mantığı

Bu projede frontend ile backend arasındaki veri iletişimi, **RESTful API** mimarisi kullanılarak gerçekleştirilmiştir. API yapısı sayesinde kullanıcı arayüzü ile veritabanı arasında doğrudan bir bağlantı kurulmamış, tüm veri alışverişi kontrollü ve güvenli bir şekilde backend üzerinden sağlanmıştır.

5.1 RESTful API Yaklaşımı

API yapısı REST prensiplerine uygun olarak tasarlanmıştır. Her kaynak (resource) belirli bir endpoint üzerinden yönetilmekte ve işlemler HTTP metotları aracılığıyla gerçekleştirilmektedir.

Kullanılan temel HTTP metotları:

- **GET** → Veri listeleme ve görüntüleme
- **POST** → Yeni veri ekleme
- **PUT** → Mevcut veriyi güncelleme
- **DELETE** → Veri silme

API cevapları JSON formatında frontend'e döndürülmektedir.

5.2 API Endpoint Yapısı

Projede kullanılan API endpoint'leri, belge ve kullanıcı işlemlerini kapsayacak şekilde tasarlanmıştır.

Örnek endpoint yapıları:

- **GET /api/documents** → Belgelerin listelenmesi
- **POST /api/documents** → Yeni belge eklenmesi

- **PUT /api/documents/{id}** → Belge güncelleme
- **DELETE /api/documents/{id}** → Belge silme

Bu endpoint'ler, ilgili Controller sınıfları içerisinde tanımlanmıştır.

5.3 Frontend – API – Veritabanı İletişimi

Bu bölümde, sistemdeki veri akışı somut bir örnek üzerinden açıklanmaktadır.

Örnek: Belge Listeleme İşlemi

1. Kullanıcı belge listeleme sayfasını açar.
2. Frontend tarafında JavaScript kullanılarak API'ye istek gönderilir.
3. API isteği backend'de ilgili Controller tarafından karşılanır.
4. Controller, gerekli iş mantığını uygular ve Entity Framework Core aracılığıyla veritabanından belge verilerini alır.
5. Alınan veriler JSON formatında frontend'e döndürülür.
6. Frontend, gelen verileri kullanıcı arayüzünde tablo halinde gösterir.

Bu yapı sayesinde:

- Veri güvenliği sağlanmıştır
- İş mantığı merkezi bir noktada toplanmıştır
- Frontend ve veritabanı birbirinden bağımsız hale gelmiştir

5.4 API Güvenliği ve Yetkilendirme

API katmanında yapılan işlemler, kullanıcı yetkilerine göre sınırlandırılmıştır. Yetkisiz kullanıcıların belirli endpoint'lere erişimi engellenmiştir.

Yetkilendirme kontrolleri:

- Kullanıcının rolü kontrol edilir
- Rol bazlı erişim uygulanır
- Kritik işlemler sadece yetkili roller tarafından yapılabilir

Bu sayede sistem, güvenli bir veri erişim yapısına sahip olmuştur.

5.5 API Yapısının Avantajları

Bu projede kullanılan API mimarisi sayesinde:

- Frontend ve backend bağımsız geliştirilebilir
- Kodun bakımı ve genişletilmesi kolaylaşır
- Güvenli ve kontrollü veri erişimi sağlanır
- Gerçek dünya web uygulamalarına uygun bir yapı oluşturulur

6. Veritabanı Tasarımı ve İlişkiler

Bu projede veritabanı, sistemde kullanılan tüm verilerin güvenli, tutarlı ve izlenebilir şekilde saklanabilmesi amacıyla **ilişkisel veritabanı yaklaşımı** kullanılarak tasarlanmıştır. Veritabanı yapısı, belge yönetim süreçlerini ve kullanıcı yetkilendirmelerini destekleyecek şekilde oluşturulmuştur.

Veritabanı işlemleri **Entity Framework Core** aracılığıyla yönetilmekte olup, uygulama ile veritabanı arasındaki iletişim nesne tabanlı bir yapı üzerinden sağlanmaktadır.

6.1 Sistemde Yer Alan Tablolar

Sistemde kullanılan veritabanı tabloları ve temel amaçları aşağıda kısaca açıklanmıştır:

- **Users**

Sistem kullanıcılarının bilgilerini tutar. Kimlik doğrulama ve yetkilendirme işlemleri bu tablo üzerinden yürütülür.

- **Roles**

Kullanıcı rollerini (*Admin, Kullanıcı, Onaycı*) tanımlar ve rol bazlı erişim kontrolünü sağlar.

- **Documents**

Sisteme eklenen belgelerin temel bilgilerini içerir. Belge adı, yükleyen kullanıcı, durum bilgisi gibi alanlar bu tabloda tutulur.

- **DocumentFiles**

Belgelerin fiziksel dosya bilgilerini ve dosya yollarını saklar. Böylece belge içeriği ile belge bilgileri ayrıştırılmıştır.

- **ApprovalRequests**

Belgeler için oluşturulan onay taleplerini tutar. Belgenin hangi aşamada olduğu bu tablo üzerinden takip edilir.

- **ApprovalActions**

Onaycı kullanıcılar tarafından yapılan onay veya red işlemlerinin kayıtlarını içerir.

- **AuditLogs**

Sistem üzerinde gerçekleştirilen önemli işlemleri kayıt altına alır. Bu yapı sayesinde işlemler geriye dönük olarak izlenebilmektedir.

- **SystemSettings**

Uygulamaya ait genel ayarları (sistem adı, dosya boyutu limiti, izin verilen uzantılar vb.) saklar.

- **__EFMigrationsHistory**

Entity Framework Core tarafından kullanılan ve veritabanı migration geçmişini tutan sistem tablosudur.

6.2 Tablolar Arası Genel İlişkiler

- Bir kullanıcı birden fazla belge yükleyebilir (1–N)
- Her belge bir kullanıcıya aittir
- Belgeler onay süreçleriyle ilişkilidir
- Onay işlemleri kullanıcı ve belge ile ilişkilendirilmiştir

Bu ilişkiler sayesinde sistemde **veri bütünlüğü, güvenlik ve izlenebilirlik** sağlanmıştır.

6.3 Veritabanı Yaklaşımının Avantajları

Bu veritabanı tasarımı sayesinde:

- Belge ve kullanıcı süreçleri net şekilde ayrılmıştır
- Onay ve loglama mekanizmaları desteklenmiştir
- Sistem ölçeklenebilir ve sürdürülebilir hale getirilmiştir

7. Uygulamanın Çalışma Senaryosu

Bu bölümde, sistemin çalışma mantığı **kullanıcı arayüzünden başlayarak veritabanına kadar** uçtan uca bir senaryo üzerinden açıklanmaktadır.

Böylece uygulamadaki veri akışı ve bileşenlerin birbiriyle nasıl etkileşim kurduğu somut olarak gösterilmektedir.

7.1 Kullanıcı Girişi ve Yetkilendirme

Kullanıcı sisteme giriş yaptığında kimlik doğrulama işlemi **ASP.NET Identity** altyapısı kullanılarak gerçekleştirilir. Kullanıcının rol bilgisi kontrol edilir ve sahip olduğu yetkilere göre erişebileceği sayfalar belirlenir.

Bu aşamada:

- Kullanıcı bilgileri **Users** tablosundan doğrulanır
- Rol bilgisi **Roles** tablosu üzerinden kontrol edilir
- Yetkisiz erişimler engellenir

7.2 Belge Yükleme Süreci

Yetkili bir kullanıcı, sistem üzerinden yeni bir belge yüklediğinde aşağıdaki adımlar gerçekleşir:

1. Kullanıcı belge yükleme formunu doldurur ve belgeyi sisteme gönderir.
2. Frontend tarafından gönderilen istek API aracılığıyla backend'e iletilir.
3. Backend, kullanıcının yetkisini kontrol eder.
4. Belgeye ait temel bilgiler **Documents** tablosuna kaydedilir.
5. Belgenin dosya bilgileri **DocumentFiles** tablosunda saklanır.
6. Belge durumu varsayılan olarak **Beklemede** olarak belirlenir.

Bu işlem sonrasında belge onay sürecine hazır hale gelir.

7.3 Belge Onay / Reddetme Süreci

Onay yetkisine sahip kullanıcılar (Admin veya Onaycı), beklemede olan belgeleri sistem üzerinden inceleyebilir.

Bu süreçte:

1. Onaycı kullanıcı belgeyi görüntüler.
2. Onay veya red işlemi seçilir.
3. Yapılan işlem **ApprovalActions** tablosuna kaydedilir.
4. Belgenin durumu **Documents** tablosunda güncellenir.
5. Onay süreci **ApprovalRequests** tablosu üzerinden takip edilir.

Bu yapı sayesinde her belge için onay geçmişi izlenebilir hale gelmiştir.

7.4 Belge Listeleme ve Filtreleme

Kullanıcılar, yetkilerine bağlı olarak:

- Tüm belgeleri
- Kendi yükledikleri belgeleri
- Belirli durumdaki belgeleri

listeleyebilmektedir.

Bu işlem sırasında:

- Frontend, API üzerinden belge listesi talep eder
- Backend, gerekli filtreleri uygular
- Veriler JSON formatında frontend'e döndürülür
- Kullanıcı arayüzünde tablo halinde gösterilir

7.5 Loglama ve Sistem İzlenebilirliği

Sistem üzerinde gerçekleştirilen önemli işlemler (belge yükleme, onay, silme vb.) **AuditLogs** tablosuna kaydedilmektedir. Bu sayede:

- Kullanıcı işlemleri izlenebilir hale gelmiştir
- Geriye dönük denetim yapılabilir
- Sistem güvenliği artırılmıştır

7.6 Senaryo Özet Akışı

Özetle uygulama aşağıdaki akış ile çalışmaktadır:

Kullanıcı → Frontend → API → İş Mantığı → Veritabanı → API → Frontend → Kullanıcı

Bu yapı sayesinde sistem:

- Güvenli
- Kontrol edilebilir
- Modüler bir yapıya sahip olmuştur.

8. Sonuç ve Değerlendirme

Bu proje kapsamında, kurum içi doküman süreçlerinin dijital ortamda güvenli, izlenebilir ve yetkilendirme temelli bir yapı ile yönetilebildiği web tabanlı bir **Belge Yönetim Sistemi (DMS)** geliştirilmiştir. Sistem; kullanıcı, rol, belge ve onay süreçlerini kapsayan bütünleşik bir mimari ile tasarlanmıştır.

Geliştirilen uygulama sayesinde:

- Kullanıcıların rol bazlı yetkilendirme ile sisteme erişimi sağlanmış,
- Belgelerin yüklenmesi, görüntülenmesi ve onay süreçleri kontrollü bir şekilde yönetilmiş,
- API, frontend ve veritabanı arasındaki veri akışı net ve güvenli bir biçimde kurgulanmıştır.

Katmanlı mimari yaklaşımı, RESTful API kullanımı ve Entity Framework Core entegrasyonu sayesinde proje; okunabilir, sürdürülebilir ve genişletilebilir bir yapıya kavuşmuştur. Ayrıca loglama ve onay mekanizmaları ile sistemin denetlenebilirliği artırılmıştır.

Bu proje ile modern web uygulamalarında kullanılan temel mimari yapıların, veri yönetimi yaklaşımlarının ve güvenlik mekanizmalarının uygulamalı olarak öğrenilmesi hedeflenmiş ve başarıyla gerçekleştirilmiştir. Geliştirilen sistem, gerçek hayatta kullanılabilecek kurumsal belge yönetim uygulamaları için sağlam bir temel oluşturmaktadır.